ITT 036 - Programming Assignment 1

Digital Signal Encoder and Decoder - Autumn 2026

Student: Shivam Kushwaha **Enrollment:** 2024BITE043

Language: Python 3 **Libraries:** numpy, matplotlib (visualization)

Objective

Develop a Python application (**signal_codec.py**) that generates digital bit streams, finds the longest palindromic substring, implements digital line encoding and decoding, and demonstrates analog-to-digital conversion with Pulse Code Modulation (PCM) and Delta Modulation (DM).

Implemented Features

- **Data generation and palindrome:** completely random bits, or random bits with fixed runs of eight and four zeros, or user-typed bits; longest palindrome found with Manacher's algorithm, O(n).
- **Line coding and decoding:** NRZ-L, NRZ-I, Manchester, Differential Manchester and AMI, each with a decoder.
- **Scrambling:** when AMI is chosen the program asks whether scrambling is needed, then B8ZS or HDB3, with decoders. Scrambling is a single O(n) pass over zero runs.
- **Analog input:** sine wave or typed samples. PCM uses n bits per sample over [-A, A]; DM uses a fixed step size. Both plot the decoder staircase and can feed the bits to any line code.
- **Visualization:** matplotlib step plots of every encoded signal (AMI shown beside the scrambled signal) and of the reconstructed analog signal.
- **Extra credit decoder:** after encoding the program asks whether to decode; decoders read only the sampled waveform, never the original bits.

Design and Assumptions

Signals are arrays of voltage levels (+1, 0, -1) at 2 samples per bit. NRZ-L: 0 = high, 1 = low. NRZ-I: 1 inverts at the start of the bit. Manchester: 1 = low-to-high, 0 = high-to-low. Differential Manchester: always a mid-bit transition, 0 adds a transition at the start of the bit. The line level before the first bit is -1, and the first AMI pulse is +1. B8ZS replaces 8 zeros by 000VB0VB. HDB3 replaces 4 zeros by 000V (odd pulse count since the last substitution) or B00V (even). Scheme choice is done through dictionaries, not long if/else chains.

How to Run

1. Install Python 3 and run `pip install numpy matplotlib`.

2. Open the project folder in a terminal and run `python signal_codec.py`; follow the menus (digital or analog, scheme, scrambling, decode).

3. Optional: `python signal_codec.py --selftest` runs the tests; `--demo DIR` saves sample plots.

Testing and Verification

The self-test runs encode-then-decode round trips for all seven schemes on over 600 sequences (including long zero runs), checks that B8ZS/HDB3 output never keeps 8/4 consecutive zero levels, compares the palindrome finder with brute force on 200 strings, and checks PCM/DM reconstruction. All passed. These results cover the tested inputs only.

Acknowledgement and References

- Help taken: AI tool (Claude, Anthropic) was used to help write the code. Add any group members or other help here.
- Forouzan, *Data Communications and Networking*; Stallings, *Data and Computer Communications*; Manacher, JACM 1975 (palindrome algorithm); ITT 036 Programming Assignment 1 specification, Autumn 2026.